

Facial Biometric Verification System - RDS

Banele Mdluli

February 2026

1 Summary

Facial biometric verification is a technology that uses facial recognition algorithms to verify the identity of an individual based on their facial features. This technology has gained popularity in recent years due to its convenience and security benefits. The facial biometric verification system can be used in various applications, including access control, identity verification, and fraud prevention. The document outlines the design specifications for facial biometric verification system.

2 Introduction

Authentications and verifications are essential in various industries, including banking, healthcare, and government. Traditional methods of authentication, such as passwords and PINs, are vulnerable to hacking and identity theft. Facial biometric verification provides a more secure and convenient alternative to traditional methods. The system uses advanced algorithms to analyze facial features and compare them to a database of known faces to verify the identity of an individual. The design specifications contains high level and low-leved design for the facial biometric verification system, including the system architecture, data flow, and user interface design.

3 Problem statement

To design an application for facial biometric verification that can be used to authenticate users based on their ID number and photo. The application should be able to integrate with other customer facing applicaitons.

4 Objectives

The project objectives are targets that must be reached for the project to be considered a success. The objectives are:

- Develop a facial biometric system to authenticate users based on their ID number and photo.
- Develop a system that integrates with other applications for general use and enhanced validation process.

5 Design

The low-level design shows the structure and make-up of individual units. The LLD is a platform divided; units generated or developed on the same platform are part of the same LLD design.

6 High technical requirements

6.1 Fast API integrations

6.1.1 Postgres, Agent and Fast API

A PostgreSQL database should be used to store information analysed in Databricks. The information stored in the database will vary from data accuracy to validity checks.

- Create an Azure Postgres table to store data quality outcomes. Depending on the outcome size, one table can be used, or multiple tables (one check per table: if required).
- Create a Fast API parent endpoint that will interact with an agent or agents that will create a data quality check PDF report.
- Create a Decision matrix that should assist the agent in compiling the report to give a certain outcome.

6.1.2 Gradio, React and Fast API

- Create an agent that should interact with an external user and provide the desired outcome based on the information available on the PostgreSQL database. The agent should interact with the database using Fast API.
- Create a Gradio chat interface that is connected to the AI agent. Link the Gradio chat app to a React Interface using Fast API.
- Create a React platform that has the following:
 - Log in page with an option to sign up.
 - A page that shows operation indicators.
 - A page that shows KPI based performance indicators.
 - A page that allows users to interact with agents.
- Report data should be downloadable in various formats (csv, tsv, etc,...).

6.2 Databricks integrations

- Create a service principal in Azure and provide it Azure blob storage role.
- Assign the same service principal a role in the Azure Databricks workspace (contributor).
- Create tables and schemas that adhere to the data engineering architecture.
- Create an API that should be used to push data from the gold layer to an external PostgreSQL.